Meeting 11. Neurons as Matrices

Act 0 Array operations from Meeting 10 · Act 1 Layer weights, scores, and probabilities · Act 2 PyTorch and GPU computation

Each **neuron** in this meeting computes a weighted sum of an input point's coordinates and adds a bias. The result is its **score**. Collecting several neurons' weights in a matrix lets us compute their scores for a batch of points in one matrix product. We then apply the sigmoid or softmax, implement the computations in PyTorch, and compare CPU and GPU timings.

The examples classify days by season from their high and low temperatures, and assign image pixels to reference colours. The image is group 7's *The Bedroom* by Vincent van Gogh. Local measurements come from a machine with 32 logical processors and no GPU; GPU measurements come from the class's discussion posts.

Notebooks, slides and reference pages.

Each notebook opens in Google Colab. Choose *Save a copy in Drive* in the *File* menu before you change anything, to save your work.

- The class notebook — the two Colab blocks of the meeting, one for each act
- Act 1: A Layer of Neurons Is a Matrix — the full notebook, Steps 1 to 8
- Act 2: The Same Layer in PyTorch, and on a GPU — the full notebook, Steps 9 to 15; it asks for a GPU in its first step
- CPU and GPU — the two processors, what is inside a core, how one product is divided, and the two memories

The four slide decks of the meeting, each available as a web page with explanatory notes and as a PDF:

- Meeting 10, Revisited — Act 0 (PDF)
- A Layer of Neurons Is a Matrix — Act 1 (PDF)
- The Same Layer in PyTorch, and on a GPU — Act 2 (PDF)
- CPU and GPU — the companion deck of Act 2 (PDF)

The main points

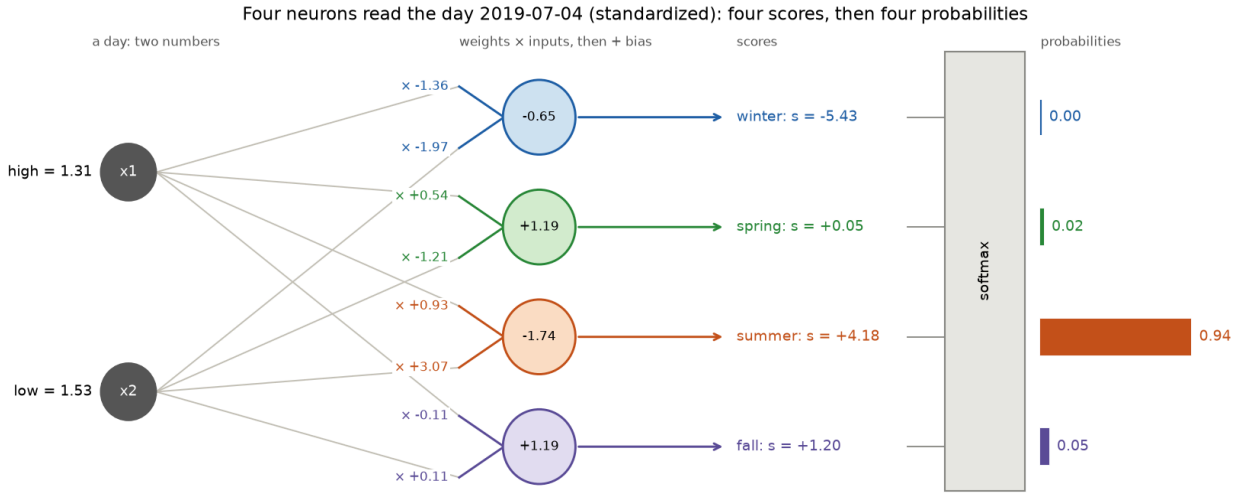
Each point below links to its explanation and code.

1 A neuron converts a point into a score

A neuron has one **weight** for each coordinate of a point and one more number, its **bias**. For a point x with m coordinates, its score is

$$s = w_1x_1 + w_2x_2 + \ \ldots\ + w_mx_m + b = w^Tx + b\,.$$

For a threshold decision, we say the neuron **fires** when $s \geq 0$, or equivalently when $w^Tx \geq \theta$, with **threshold** $\theta = -b$. With w and x written as columns, w^Tx is a row-column product. NumPy represents them here as one-dimensional arrays of shape `(m,)`, so `w @ x + b` computes their dot product plus the bias. The season example uses weights obtained by **training**: adjusting the parameters using labelled days from 2006 to 2017. Its prediction selects the largest of four scores.

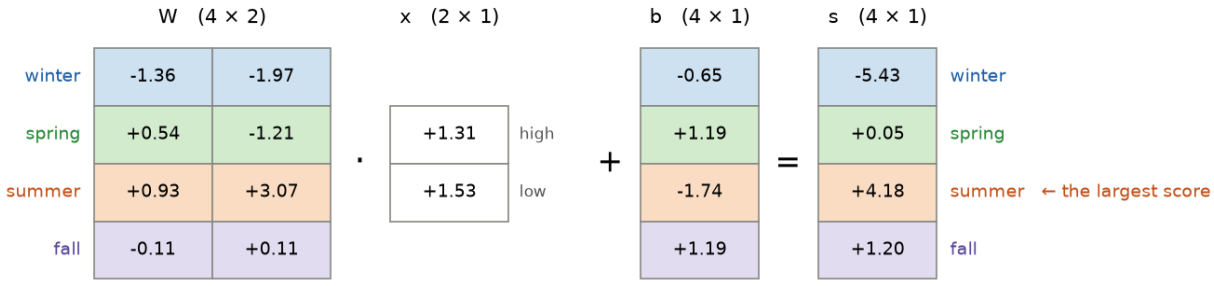


Scores and softmax probabilities for July 4, 2019.

Sections 1.2 and 1.6.

2 Each row of the weight matrix represents one neuron

A **layer** contains neurons that receive the same input. For K neurons with m input coordinates, the weight matrix W has shape $K \times m$. Row j contains neuron j's weights; a separate vector b contains the K biases. The score vector for one point is $s = Wx + b$, of shape $K \times 1$ in column notation. When the neurons represent possible classes, the predicted class is the index of the largest score, **argmax**.



Four neurons and one day: s = Wx + b, with the summer neuron's score the largest.

Section 1.3.

3 Matrix multiplication computes scores for a batch

Store n points as the rows of X, an $n \times m$ matrix. The scores of all K neurons on all n points form an $n \times K$ matrix,

$$S = XW^T + 1\,b^T,$$

NumPy writes this as `X @ W.T + b`. Entry $S[i, j]$ is neuron j's score for point i. Transposing W places each neuron's weights in a column for the matrix product. The term $1\,b^T$ adds the biases to every row; NumPy performs this addition by **broadcasting**. Transposition reverses a product's factors: $(AB)^T = B^T A^T$. Matrix multiplication is generally not commutative.

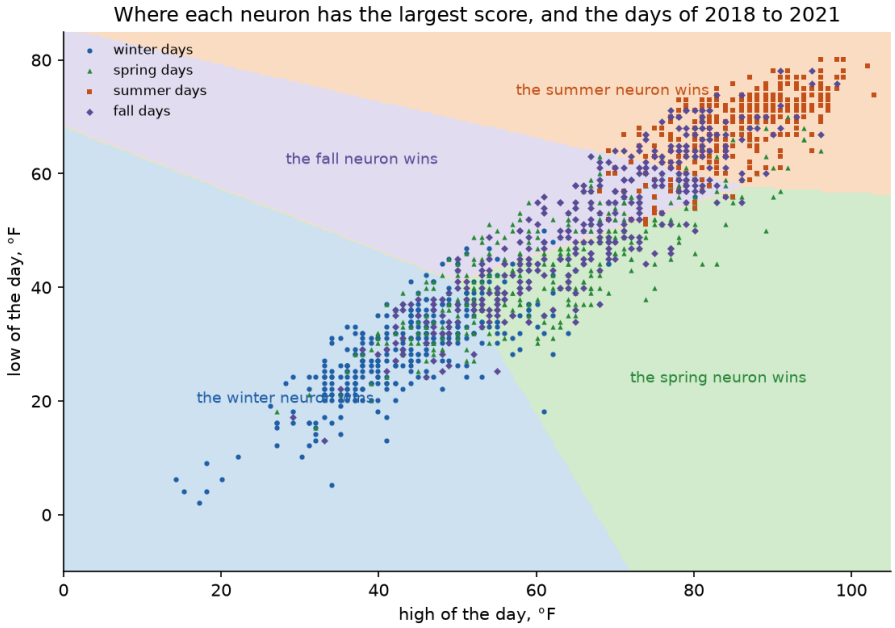
	one point `x`, shape `(m,)`	n points `X`, shape `(n, m)`
one neuron: `w` of shape `(m,)`, `b` a number	`w @ x + b` is a number	`X @ w + b` has shape `(n,)`
K neurons: `W` of shape `(K, m)`, `b` of shape `(K,)`	`W @ x + b` has shape `(K,)`	`X @ W.T + b` has shape `(n, K)`

Compatible shapes do not guarantee the intended computation. When $K = m$, `X @ W + b` runs, but generally gives different scores from `X @ W.T + b`.

Sections 1.5 and 1.8.

4 Linear scores produce straight decision boundaries

The season classifier predicted correctly on 59.3% of the 1,457 days from 2018 to 2021, which were excluded from training. Two neurons have equal scores where $(w_j - w_k)^T z + (b_j - b_k) = 0$. With two input coordinates and distinct weight vectors, this equation defines a line. Standardization shifts and scales coordinates, so the boundary remains straight when plotted in degrees. A class's decision region is where its score is at least as large as every other score. Spring and fall temperatures overlap substantially in the plot, making those seasons difficult to distinguish from these two measurements.



Where each neuron has the largest score, and the days of 2018 to 2021 in the colours of their true seasons.

Section 1.5.

5 Sigmoid acts entry by entry; softmax normalizes each row

The **sigmoid** $\sigma(s) = 1 / (1 + e^{-s})$ maps each score separately to a number between 0 and 1, and its result has the shape of its input. The **softmax** maps the K scores of one point to K positive numbers that add to 1,

$$p_k = \frac{e^{s_k}}{e^{s_1} + e^{s_2} + \ \ldots \ + e^{s_K}},$$

Each softmax probability depends on every score in its row. Subtracting the row maximum leaves the mathematical result unchanged and prevents exponential overflow. These normalized values serve as model probabilities or mixture weights; normalization alone does not establish their accuracy as probabilities of real outcomes. A **temperature** $T > 0$ rescales the scores before softmax. Lower temperatures concentrate weight on the largest scores; higher temperatures give a more uniform distribution. For the day in point 1:

OUTPUT

```
the four scores:        winter -5.43, spring +0.05, summer +4.18, fall +1.20
their probabilities:    winter 0.000, spring 0.015, summer 0.937, fall 0.048   (they add to 1.000)
sigmoid of each score:  winter 0.004, spring 0.513, summer 0.985, fall 0.769   (each entry by itself)
```

Sections 1.4, 1.9 and 1.10.

6 Nearest-colour assignment can use linear scores

Give neuron j the weights c_j, a reference colour, and the bias $-\frac{1}{2}\|c_j\|^2$, so that its score is $s_j = c_j^T x - \frac{1}{2}\|c_j\|^2$. Expanding the squared distance gives

$$\|x - c_j\|^2 = \|x\|^2 - 2\left(c_j^T x - \tfrac{1}{2}\|c_j\|^2\right) = \|x\|^2 - 2s_j\,.$$

For a fixed pixel, $\|x\|^2$ is the same for every reference colour. Maximizing the score therefore minimizes squared distance in RGB coordinates. The notebook supplies five reference colours computed by **k-means**, a clustering method that groups pixels around representative centres. Each row of W contains one reference colour's red, green, and blue values.

W (5 × 3): row j, in its own colour

	red	green	blue
neuron 0	0.32	0.36	0.31
neuron 1	0.63	0.46	0.10
neuron 2	0.53	0.53	0.45
neuron 3	0.48	0.57	0.66
neuron 4	0.67	0.65	0.55

The weights of the colour layer: row j of W is reference colour j.

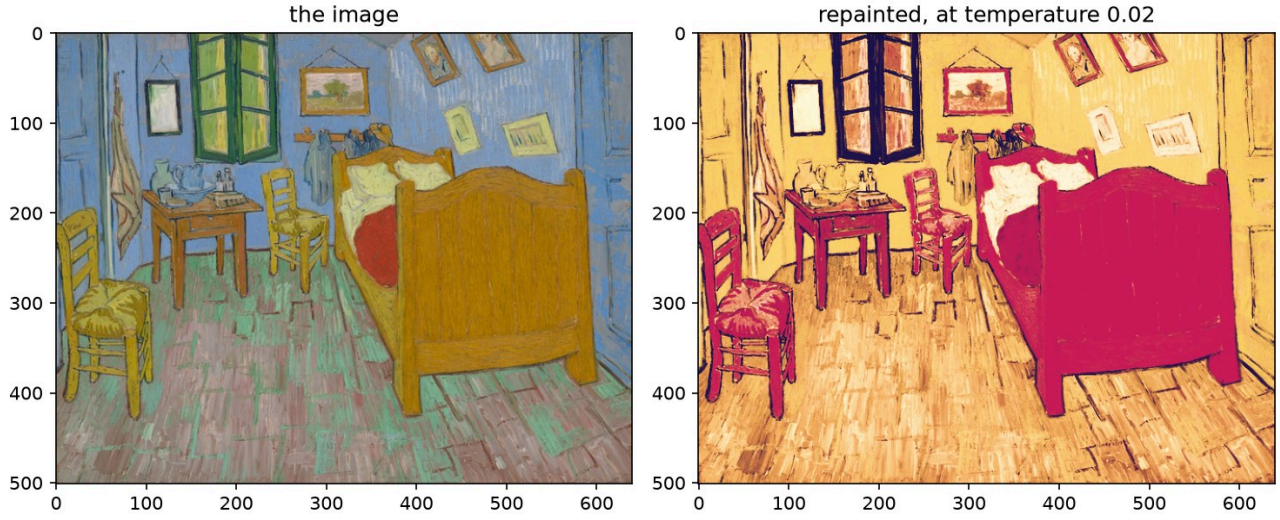
Section 1.7.

7 A second layer mixes colours using softmax weights

Assign an output colour, or *paint*, to each of the K neurons. Store these colours as the rows of a $K \times 3$ matrix `PAINT`. Then

```
Prob = softmax_rows(layer(X, W, b) / T)    # n x K probabilities
new = Prob @ PAINT                         # n x 3 colours
```

computes a weighted average of the paints for each pixel. The second matrix product maps K probability weights to three output channels. It is another linear layer, with weight matrix `PAINT.T` and zero bias. Together, the two layers and the intervening softmax form a small **neural network** with fixed parameters. As $T \to 0$, a pixel with a unique largest score approaches that neuron's paint. As $T \to \infty$, all weights approach $1 / K$, so every pixel approaches the average paint colour.



The Bedroom, repainted at temperature 0.02 with the notebook's five paints.

Section 1.10.

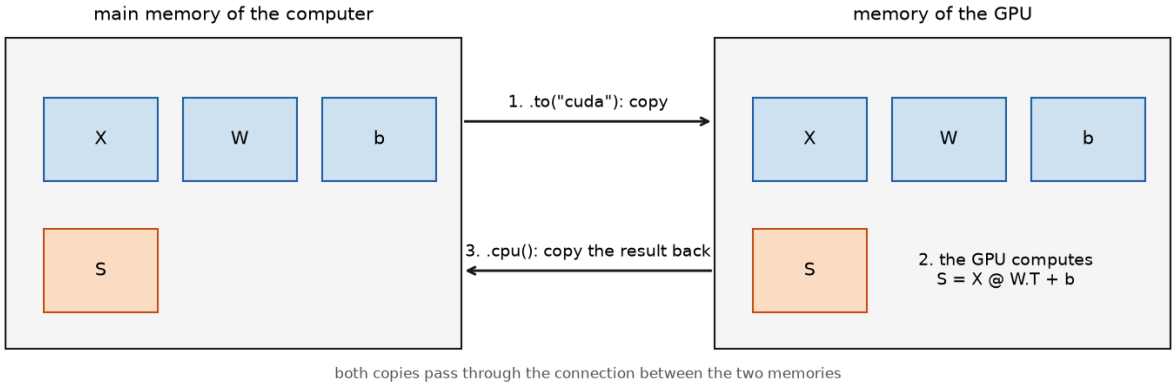
8 PyTorch implements the layer with tensor operations

A PyTorch **tensor** is a multidimensional array. For the matrices here, `X @ W.T + b` uses the same multiplication, transpose, and broadcasting rules as NumPy. Floating-point tensors created from Python floats default to `float32` unless configured otherwise; the NumPy arrays in these examples use `float64`. The matrix multiplication requires matching data types. PyTorch reductions use `dim` and `keepdim`, and `S.max(dim=1)` returns both values and indices. Its built-in layer, `torch.nn.Linear(m, K)`, stores weights of shape `(K, m)` and computes the same affine transformation.

Sections 2.1 to 2.4.

9 Tensor placement determines where computation runs

A tensor's **device** identifies where its data are stored. In these examples, that is CPU memory or an NVIDIA GPU's memory. Moving a CPU tensor with `.to("cuda")` copies its data to the GPU; `.cpu()` copies a GPU result back. The tensors in `X @ W.T + b` must be on the same device. GPU operations are usually asynchronous: Python can continue before they finish. When timing with a CPU clock, synchronize before starting and before stopping the timer.

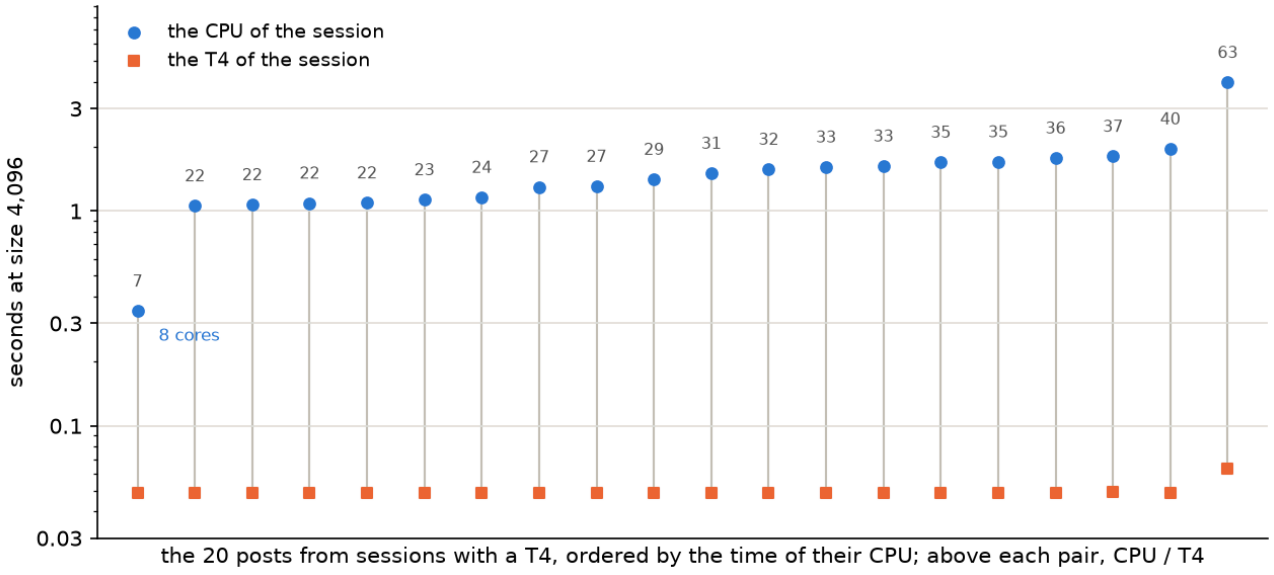


Two memories: the copies to the GPU and back pass through the connection between them.

Sections 2.5 and 2.6.

10 The class measured substantial T4 speedups on large products

The output entries of S can be computed independently, giving matrix multiplication substantial parallel work. The T4 has 2,560 CUDA arithmetic cores, organized differently from CPU cores. In the class's measurements, `X @ W.T + b` with $n = m = K = 4096$ took 0.0486 to 0.0490 seconds on the T4 in 19 of 20 sessions. Two-core CPU times ranged from 1.06 to 1.95 seconds, with one slower session. The median CPU-to-GPU time ratio was 31 at size 4,096 and 16 at size 512. These timings exclude transfers between CPU and GPU memory.



The class's measurements at size 4,096: the CPU and the T4 of each session, with the factor between them.

Section 2.7.

Working with the notebooks and an AI assistant.

Run each notebook from the top with Shift+Enter. Complete each **Your turn** exercise, then run its check. In the full notebooks, Step 6 of Act 1 asks an AI assistant to write `softmax_rows`, and Step 11 of Act 2 asks for its PyTorch translation. Record the request in **Our prompt**, specifying the function's name, input, output, and restrictions. The shorter class notebook supplies the first function and asks you to implement the matrix expression for the layer.

Act 0. Array operations from Meeting 10

The following array operations support the layer computations. Their explanations and code are in the Meeting 10 notes.

- **Vectorization.** Array expressions move repeated arithmetic from a Python loop into NumPy's compiled operations. The Meeting 10 experiments compare their running times. Meeting 10, point 2.
- **Broadcasting.** `X - mu` subtracts m numbers from every row of an $n \times m$ array without storing n copies. Shapes are compared from the right, and a missing dimension, or one of length 1, is repeated. In this meeting broadcasting adds the biases of a layer. Idea 4.
- **Slicing.** `X[0]` is a row, `X[:, 0]` a column, `image[:, :, 0]` a channel. In this meeting, `S[:, j]` holds the scores of neuron j. Idea 5.
- **Axes.** A reduction such as `X.mean(axis=0)` combines entries along the named dimension. By default that dimension disappears; `keepdims=True` retains it with length 1. Here, `S.max(axis=1)` and `S.argmax(axis=1)` find each row's largest score and its index. Point 6.
- **Word vectors.** `U @ v` computes a dot product between `v` and each of 50,000 word vectors. This has the form of one neuron with weights `v` and bias zero, applied to 50,000 points. Point 7.
- **Batches.** N points of the same shape are stored in one array whose first dimension counts the points. Idea 8.

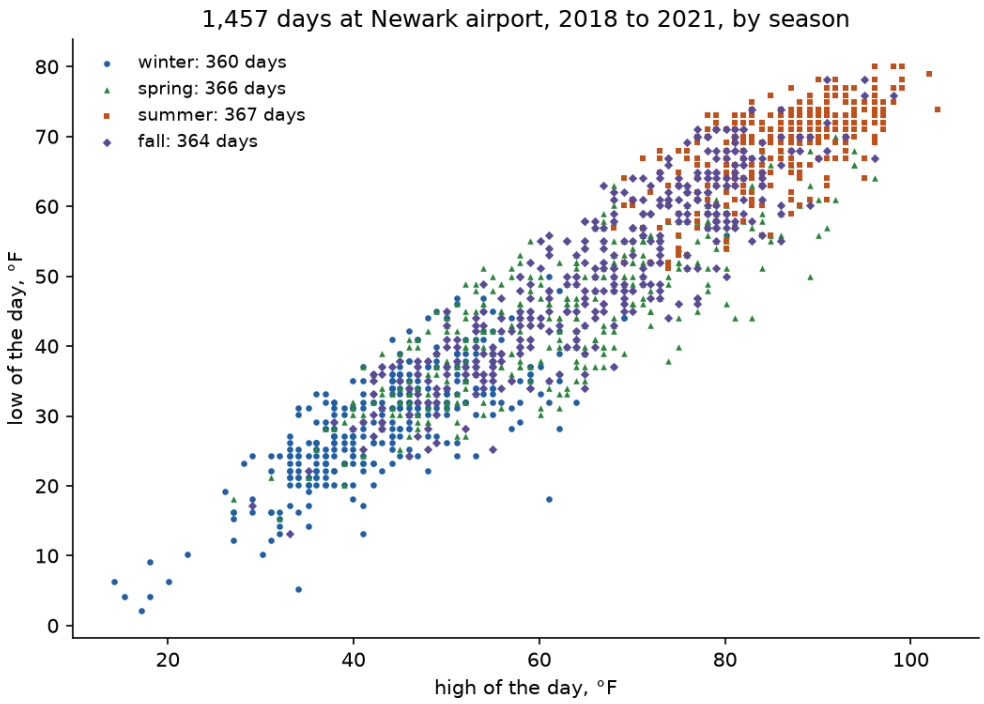
The data points, neuron weights, and scores can all be represented as arrays.

Act 1. Layer weights, scores, and probabilities Open the notebook of Act 1

We first classify days by season, then apply the same matrix operations to image pixels. Section headings identify the corresponding notebook steps; the weather example is in Step 8.

1.1 The data: days at Newark airport Step 8

The weather dataset covers Newark airport from 2006 to 2024. Each **point** is a day with two coordinates: its high and low temperatures in °F. The evaluation set contains 1,457 days from 2018 to 2021, stored as matrix rows. The notebook standardizes each coordinate using fixed means and standard deviations. A day's **label** is its season, assigned by month: winter is December-February, spring March-May, summer June-August, and fall September-November.



The days of 2018 to 2021 by their high and low temperatures, coloured by season.

Code: the days, their labels, and the trained layer of four neurons

cells `seasons`, `seasons-data`

```
weather = pd.read_csv(SITE + "weather/newark-weather.csv")
# the days with no missing measurement
weather = weather.dropna(subset=["tmax_f", "tmin_f", "humidity_pct", "pressure_hpa"])
days = weather[weather["date"].str[:4].isin(["2018", "2019", "2020", "2021"])]
month = days["date"].str[5:7].astype(int).to_numpy()
season = (month % 12) // 3               # 0 winter (December to February), 1 spring, 2 summer, 3 fall

X_days = days[["tmax_f", "tmin_f"]].to_numpy()              # the high and the low of each day
mu, sd = np.array([64.7888, 47.9889]), np.array([18.5731, 16.9396])
Z_days = (X_days - mu) / sd                                 # standardized, as in meeting 10

W_seasons = np.array([[-1.35750, -1.96665],      # the neuron of winter
                      [ 0.54402, -1.21157],      # the neuron of spring
                      [ 0.92758,  3.07303],      # the neuron of summer
                      [-0.11410,  0.10518]])     # the neuron of fall
b_seasons = np.array([-0.64726, 1.19295, -1.73890, 1.19321])

S_days = Z_days @ W_seasons.T + b_seasons        # the line of Step 4
print("shape of Z_days:", Z_days.shape, "  shape of W_seasons:", W_seasons.shape)
print("shape of S_days:", S_days.shape)
right = (S_days.argmax(axis=1) == season).mean()
print(f"The neuron with the largest score names the right season on {right:.1%} of the {len(days):,} days.")
```

OUTPUT

```
shape of Z_days: (1457, 2)   shape of W_seasons: (4, 2)
shape of S_days: (1457, 4)
The neuron with the largest score names the right season on 59.3% of the 1,457 days.
```

OUTPUT

```
The daily weather at Newark airport, 6,865 days from 2006 to 2024: the high and the low of each day, in °F.
The days of 2018 to 2021 are n = 1,457 points with m = 2 coordinates each. The first four:
            date  tmax_f  tmin_f
2018-01-01    20.1     6.3
2018-01-02    27.1    12.2
2018-01-03    30.2    10.2
2018-01-04    29.1    18.1
The label of a day is its season, from its month: winter is December to February, then spring, summer, fall.
```

1.2 One neuron, one day: the score by hand

The winter neuron uses two weights and a bias. On July 4, 2019, the high was 89 °F and the low 74 °F, giving standardized coordinates $z = (1.31, 1.53)$. Its score is the weighted sum plus the bias:

OUTPUT

```
Day 2019-07-04: high 89 °F, low 74 °F. Standardized: z = (1.31, 1.53).
The winter neuron: w = (-1.36, -1.97), b = -0.65.
Its score: -1.36 x 1.31 + (-1.97) x 1.53 + (-0.65) = -5.43
shape of w: (2,), shape of z: (2,); w @ z + b is one number.
```

The negative score means this neuron does not fire under the threshold rule. In column notation, w and z have shape 2×1, so $w^T z$ has shape 1×1. Multiplying the two columns without transposing is undefined. NumPy instead stores these vectors as arrays of shape `(2,)`, for which `@` computes the dot product.

1.3 Computing four season scores

The four season neurons share the same input. Their weights form a 4×2 matrix W, and their biases form a four-entry vector b. The expression $Wz + b$ computes their scores:

OUTPUT

```
W @ z + b, shape (4,): winter -5.43, spring +0.05, summer +4.18, fall +1.20
shape of W: (4, 2), shape of b: (4,)
The largest score is that of the summer neuron, at position 2: the argmax.
```

In column notation, Wz and b both have shape 4×1, so the addition needs no broadcasting. The summer neuron has the largest score, at index 2. The classifier therefore predicts summer.

Code: one neuron and four neurons on one day — cell `seasons-one-day`

```python
DAY = "2019-07-04"
if not (days["date"] == DAY).any():
    DAY = days["date"][days["date"].str.startswith("2019-07")].iloc[0]
k_day = int(np.flatnonzero(days["date"].to_numpy() == DAY)[0])
x_day = X_days[k_day]                       # the high and the low of the day, in °F
z_day = Z_days[k_day]                       # the same day, standardized
w_winter, b_winter = W_seasons[0], b_seasons[0]
s_winter = w_winter @ z_day + b_winter
print(f"Day {DAY}: high {x_day[0]:.0f} °F, low {x_day[1]:.0f} °F. Standardized: z = ({z_day[0]:.2f}, {z_day[1]:.2f}).")
print(f"The winter neuron: w = ({w_winter[0]:.2f}, {w_winter[1]:.2f}), b = {b_winter:.2f}.")
print(f"Its score: {w_winter[0]:.2f} x {z_day[0]:.2f} + ({w_winter[1]:.2f}) x {z_day[1]:.2f} + ({b_winter:.2f}) =
{s_winter:.2f}")
print(f"shape of w: {w_winter.shape}, shape of z: {z_day.shape}; w @ z + b is one number.")
```

```
s_day = W_seasons @ z_day + b_seasons      # the four scores of the day: one line
print(f"W @ z + b, shape {s_day.shape}: " + ", ".join(f"{name} {value:+.2f}" for name, value in zip(SEASONS, s_day)))
print(f"shape of W: {W_seasons.shape}, shape of b: {b_seasons.shape}")
print(f"The largest score is that of the {SEASONS[int(s_day.argmax())]} neuron, at position {int(s_day.argmax())}: the
argmax.")
```

1.4 From scores to probabilities: the softmax and the sigmoid Step 6

Softmax assigns the day the following model probabilities: summer 0.937, fall 0.048, spring 0.015, and winter 0.000. Applying sigmoid separately to each score does not generally produce values that sum to 1. For example, the spring score is 0.05, whose sigmoid is 0.513, close to one half.

Step 6 asks for `softmax_rows` to accept an $n \times K$ NumPy array and return an array of the same shape, without loops or a library softmax. `S.max(axis=1, keepdims=True)` returns the row maxima with shape $n \times 1$, allowing each maximum to be subtracted from its row. Adding a constant c to a row multiplies its exponentials by the common factor e^c, which cancels during normalization. Subtracting the maximum therefore preserves softmax while keeping exponentials at most 1. The check includes scores near 1,000 to test overflow protection.

Code: Step 6, the softmax of every row, and its check cells `paste-softmax`, `run-check-softmax`, `seasons-softmax`

```
def softmax_rows(S):
    shifted = S - S.max(axis=1, keepdims=True)       # the largest entry of every row becomes 0
    E = np.exp(shifted)
    return E / E.sum(axis=1, keepdims=True)
```

```
assert "softmax_rows" in globals(), "Paste the function softmax_rows into the cell above and run that cell."
assert not has_loop(softmax_rows), "This function must use operations on whole arrays: no loop."
assert not mentions(softmax_rows, ("scipy", "torch", "sklearn")), (
    "This function must compute the softmax itself, with NumPy only: no softmax function from a library.")

test = np.array([[-1.0, 0.0, 2.0, 1.0],
                 [0.0, 0.0, 0.0, 0.0],
                 [1000.0, 1003.0, 1001.0, 1002.0]])
expected = np.array([[0.0321, 0.0871, 0.6439, 0.2369],
                     [0.25, 0.25, 0.25, 0.25],
                     [0.0321, 0.6439, 0.0871, 0.2369]])
try:
    with np.errstate(all="ignore"):
        result = np.asarray(softmax_rows(test.copy()), dtype=float)
except ValueError:
    raise AssertionError(
        "The shapes inside softmax_rows do not fit. The largest entry and the sum of every row must keep "
        "the shape (n, 1), so that they can be combined with the n rows: use keepdims=True.") from None
assert result.shape == test.shape, f"softmax_rows returned an array of shape {result.shape}; it must be {test.shape}."
assert not np.isnan(result).any(), (
    "softmax_rows returns nan (not a number) for the row of very large scores. "
    "Subtract the largest entry of each row before the exponential.")
assert np.allclose(result.sum(axis=1), 1), (
    "The rows of the result do not add to 1. The largest entry and the sum must be taken along each row: axis=1.")
assert np.allclose(result, expected, atol=2e-4), (
    f"The result is not the softmax of the rows. For the scores -1, 0, 2, 1 the function gives {np.round(result[0], 2)};
```

```
"
    "the right values are 0.03, 0.09, 0.64, 0.24.")
print("Correct: every row of the result adds to 1, also for very large scores.")
print("The scores -1, 0, 2, 1 give", np.round(result[0], 2))
```

OUTPUT

```
Correct: every row of the result adds to 1, also for very large scores.
The scores -1, 0, 2, 1 give [0.03 0.09 0.64 0.24]
```

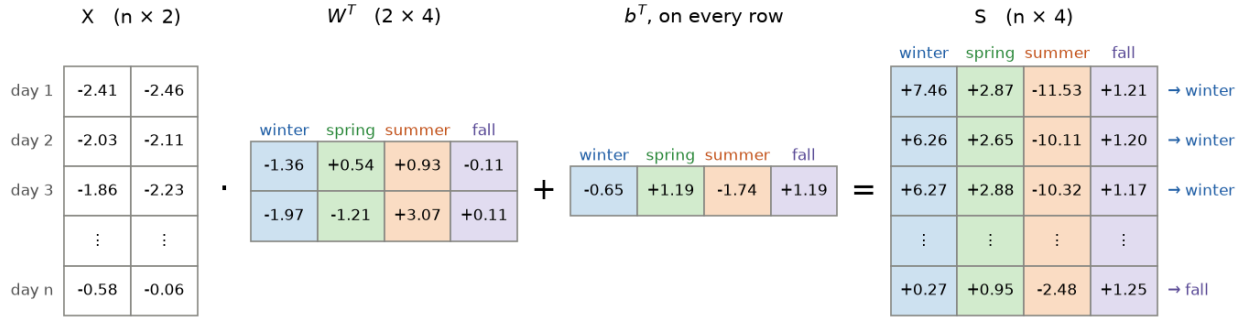
```
p_day = softmax_rows(s_day.reshape(1, 4))[0]        # the softmax of one row of four scores
print("the four scores:        " + ", ".join(f"{name} {value:+.2f}" for name, value in zip(SEASONS, s_day)))
print("their probabilities:    " + ", ".join(f"{name} {value:.3f}" for name, value in zip(SEASONS, p_day)) + f"   (they
add to {p_day.sum():.3f})")
print("sigmoid of each score:  " + ", ".join(f"{name} {value:.3f}" for name, value in zip(SEASONS, sigmoid(s_day))) + "
(each entry by itself)")
```

OUTPUT

```
the four scores:        winter -5.43, spring +0.05, summer +4.18, fall +1.20
their probabilities:    winter 0.000, spring 0.015, summer 0.937, fall 0.048   (they add to 1.000)
sigmoid of each score:  winter 0.004, spring 0.513, summer 0.985, fall 0.769   (each entry by itself)
```

1.5 Computing scores for all days

`Z_days @ W_seasons.T + b_seasons` produces an $n \times 4$ score matrix. In mathematical notation, $S = XW^T + 1b^T$, with X of shape $n \times 2$ and W^T of shape 2×4. Each row contains one day's four scores. NumPy broadcasts the bias vector across all rows; $1b^T$ expresses that repeated addition mathematically.



The scores of all the days in one product. The columns of the transposed weights and of S belong to the four neurons.

The index of each row's largest score is the predicted season. Accuracy on the evaluation days is 59.3%. Point 4 shows the decision regions and the overlap between spring and fall. Point 3 summarizes the array shapes for one or several neurons applied to one or several points.

Code: the regions of the four neurons cell `seasons-many`

```
print("shape of Z_days:", Z_days.shape, "  shape of W_seasons.T:", W_seasons.T.shape, "  shape of S_days:", S_days.shape)
named = S_days.argmax(axis=1)              # for every day, the neuron with the largest score
```

```
print(f"The largest score names the right season on {(named == season).mean():.1%} of the {len(days):,} days.")
highs, lows = np.linspace(0, 105, 211), np.linspace(-10, 85, 191)
HH, LL = np.meshgrid(highs, lows)
grid_points = np.column_stack([HH.ravel(), LL.ravel()])
region = (((grid_points - mu) / sd) @ W_seasons.T + b_seasons).argmax(axis=1).reshape(HH.shape)
fig, ax = plt.subplots(figsize=(7.6, 5.4))
ax.contourf(HH, LL, region, levels=[-0.5, 0.5, 1.5, 2.5, 3.5], colors=fills)
for k in range(4):
    sel = season == k
    ax.scatter(X_days[sel, 0], X_days[sel, 1], s=9, marker=markers[k], color=dark[k], linewidths=0, label=f"{SEASONS[k]}
days")
    inside = region == k
    if inside.mean() > 0.04:
        ax.text(HH[inside].mean(), LL[inside].mean(), f"the {SEASONS[k]} neuron wins", ha="center", va="center",
                fontsize=9, color=dark[k])
ax.set_xlabel("high of the day, °F")
ax.set_ylabel("low of the day, °F")
ax.set_title("Where each neuron has the largest score, and the days of 2018 to 2021")
ax.legend(loc="upper left", frameon=False, fontsize=8)
ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
plt.show()
```

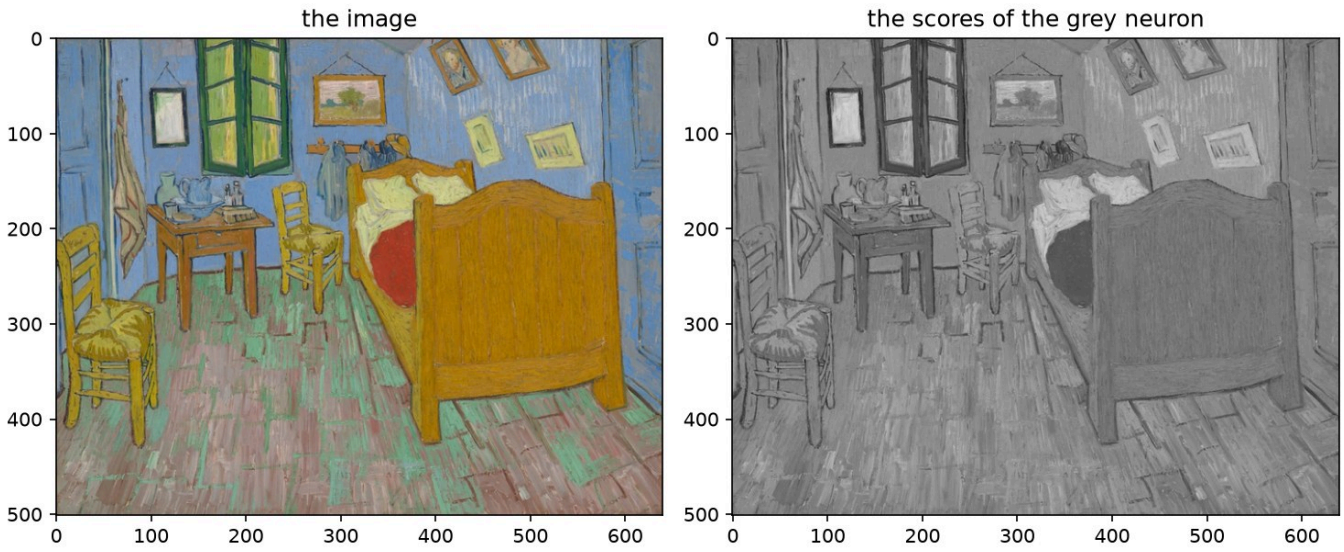
OUTPUT

```
shape of Z_days: (1457, 2)   shape of W_seasons.T: (2, 4)   shape of S_days: (1457, 4)
The largest score names the right season on 59.3% of the 1,457 days.
```

1.6 A second example: a neuron that reads a pixel Steps 1 and 2

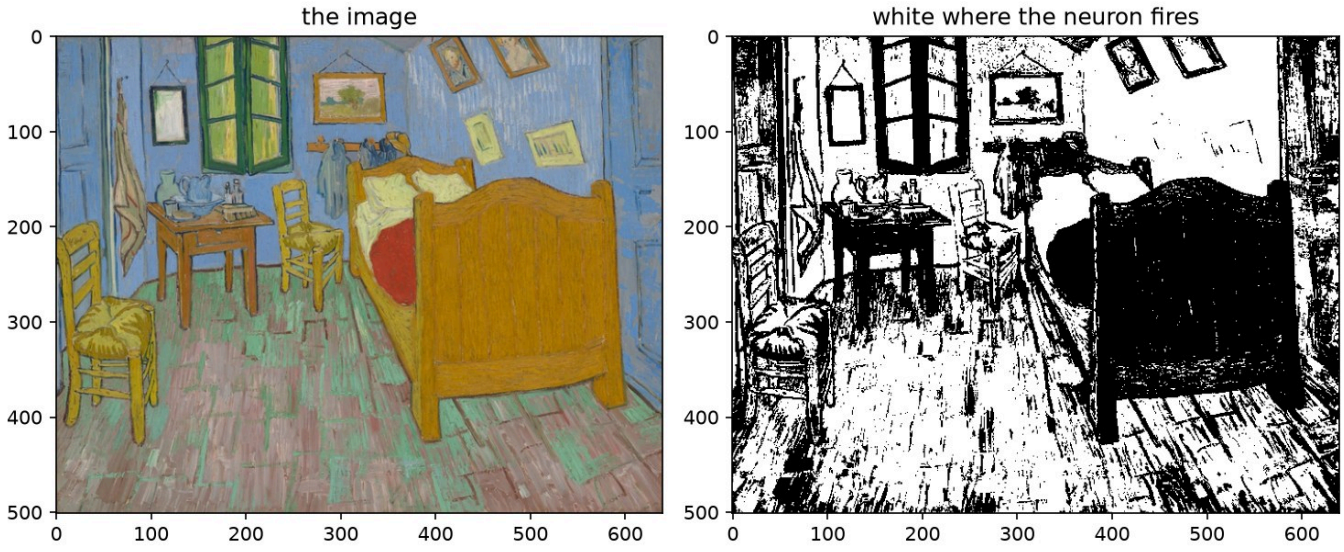
The image example treats each **pixel** as a point with three coordinates: red, green, and blue values between 0 and 1. `image.reshape(-1, 3)` arranges the pixels as rows of `X`. For *The Bedroom*, this gives 320,640 points.

The grayscale rule from Meeting 10 uses weights (0.299, 0.587, 0.114) and bias zero. We use its weighted RGB value as a brightness measure. For the middle pixel, (0.56, 0.39, 0.10), the score is 0.411; hand calculation with the rounded coordinates may differ in the third decimal place. `X @ w_grey + b_grey` evaluates the rule for every pixel. Reshaping the result to the image's height and width gives the grayscale image, with maximum difference 0.0020 from the stored Meeting 10 result.



The original image and the grayscale scores reshaped to image dimensions.

These brightness values are nonnegative, so bias zero makes the neuron fire on every pixel under the $s \geq 0$ rule. Setting the bias to -0.518, the negative mean brightness, selects pixels at least as bright as the mean: 53.1% of the image. Step 2 asks you to choose another neuron. The starting weights (1, -0.5, -0.5) and bias -0.1 select pixels whose red value exceeds the mean of green and blue by at least 0.1. Here, 36.4% of pixels satisfy that condition.



White where the grey neuron fires, with its threshold at the mean brightness.

The notebooks store data points as rows, giving the score vector $Xw + b$. A column-based convention uses X^T and writes the same scores as $w^T X^T + b$, now arranged as a row. Step 2 computes both representations and checks agreement. NumPy and PyTorch support either convention; the formulas must match the chosen layout.

Code: Steps 1 and 2, one neuron on one pixel and on all of them

cells `image-number`, `one-unit-one-point`, `transpose`, `one-unit-many-points`, `fires`, `own-unit`, `two-conventions`

```
IMAGE = 7        # replace 0 with your group's number, from 1 to 30

assert 1 <= IMAGE <= 30, "Type your group's number in place of the 0."

images = pd.read_csv(IMAGES + "images.csv", index_col="number")
picture_info = images.loc[IMAGE]
image = read_image(picture_info["file"])
height, width = image.shape[0], image.shape[1]
print(f"Image {IMAGE}: {picture_info['title']} ({picture_info['creator']})")
print(f"shape of image: {image.shape}, which is {height} rows and {width} columns of pixels")
show([image], [f"image {IMAGE}"])

x = image[height // 2, width // 2]             # the pixel in the middle of the image
w_grey = np.array([0.299, 0.587, 0.114])       # the weights of the grey neuron
b_grey = 0.0                                   # its bias
print("the pixel   x =", np.round(x, 2), " (red, green, blue)")
print("the weights w =", w_grey, " and the bias b =", b_grey)
```

OUTPUT

```
Image 7: The Bedroom (Vincent van Gogh)
shape of image: (501, 640, 3), which is 501 rows and 640 columns of pixels
the pixel   x = [0.56 0.39 0.1 ]  (red, green, blue)
the weights w = [0.299 0.587 0.114]  and the bias b = 0.0
```

```
s_one = w_grey @ x + b_grey          # the dot product of the weights and the point, plus the bias
print(f"s = {s_one:.3f}")
print("shape of w_grey:", w_grey.shape, "  shape of x:", x.shape)
```

OUTPUT

```
s = 0.411
shape of w_grey: (3,)   shape of x: (3,)
```

```
print("shape of w_grey.T:", w_grey.T.shape, "- an array with one dimension is its own transpose")

w_column = w_grey.reshape(3, 1)      # the same numbers as a matrix with 3 rows and 1 column
x_column = x.reshape(3, 1)
print("shape of w_column:", w_column.shape, "  shape of w_column.T:", w_column.T.shape)
print("w_column.T @ x_column =", w_column.T @ x_column, "- a matrix with 1 row and 1 column")
try:
    w_column @ x_column
except ValueError:
    print("w_column @ x_column fails: a matrix with 1 column cannot be multiplied by a matrix with 3 rows")
```

OUTPUT

```
shape of w_grey.T: (3,) - an array with one dimension is its own transpose
shape of w_column: (3, 1)   shape of w_column.T: (1, 3)
w_column.T @ x_column = [[0.41066667]] - a matrix with 1 row and 1 column
w_column @ x_column fails: a matrix with 1 column cannot be multiplied by a matrix with 3 rows
```

```
X = image.reshape(-1, 3)                # one row for each pixel
n, m = X.shape
print(f"shape of X: {X.shape}, which is n = {n:,} points with m = {m} coordinates")

s_grey = X @ w_grey + b_grey            # the scores of the grey neuron on all the points
print("shape of X @ w_grey + b_grey:", s_grey.shape)
print("the scores of the first five points:", np.round(s_grey[:5], 3))

grey = read_image(picture_info["grey_file"])          # the grey image of meeting 10
difference = np.abs(s_grey.reshape(height, width) - grey).max()
print(f"largest difference between the scores and the grey image of meeting 10: {difference:.4f}")
show([image, s_grey.reshape(height, width)], ["the image", "the scores of the grey neuron"])
```

OUTPUT

```
shape of X: (320640, 3), which is n = 320,640 points with m = 3 coordinates
shape of X @ w_grey + b_grey: (320640,)
the scores of the first five points: [0.482 0.532 0.5   0.482 0.477]
largest difference between the scores and the grey image of meeting 10: 0.0020
```

```
b_bright = -s_grey.mean()                       # minus the mean brightness of the image
fires = X @ w_grey + b_bright >= 0              # True for the points on which the neuron fires
print(f"bias {b_bright:.3f}: the neuron fires on {fires.mean():.1%} of the {n:,} pixels")
show([image, fires.reshape(height, width).astype(float)], ["the image", "white where the neuron fires"])
```

OUTPUT

```
bias -0.518: the neuron fires on 53.1% of the 320,640 pixels
```

```
w_own = np.array([1.0, -0.5, -0.5])     # the weights for red, green and blue
b_own = -0.1                            # the bias

fires_own = X @ w_own + b_own >= 0
print(f"weights {w_own}, bias {b_own}: the neuron fires on {fires_own.mean():.1%} of the pixels")
show([image, fires_own.reshape(height, width).astype(float)], ["the image", "white where our neuron fires"])
```

OUTPUT

```
weights [ 1.  -0.5 -0.5], bias -0.1: the neuron fires on 36.4% of the pixels
```

```
by_rows = X @ w_grey + b_bright         # points as rows: X has shape (n, m)
by_columns = w_grey @ X.T + b_bright    # points as columns: X.T has shape (m, n)
print("shape of X:", X.shape, "  shape of X.T:", X.T.shape)
print("The two results are equal:", np.allclose(by_rows, by_columns))
try:
    w_grey @ X
except ValueError:
    print(f"w_grey @ X fails: w_grey has 3 entries and X has {n:,} rows, so the shapes do not fit")
```

OUTPUT

```
shape of X: (320640, 3)   shape of X.T: (3, 320640)
The two results are equal: True
w_grey @ X fails: w_grey has 3 entries and X has 320,640 rows, so the shapes do not fit
```

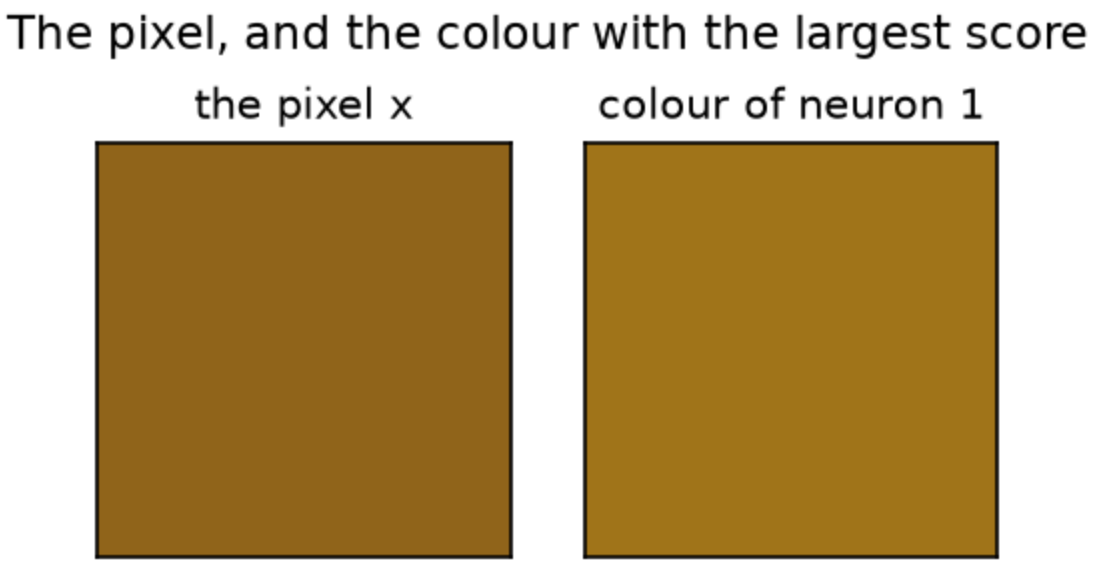
1.7 Five neurons, one for each reference colour Step 3

This layer has five neurons, one per reference colour. The setup computes these colours as cluster centres and orders them from darkest to lightest. Row j of `W` is c_j, with bias $-\frac{1}{2}\left\| c_j \right\|^2$. The identity in point 6 shows that the largest score identifies the nearest reference colour in RGB distance.

On the middle pixel, `W @ x + b` gives five scores. The largest is that of neuron 1, whose colour is the nearest of the five:

OUTPUT

```
the pixel x = [0.56 0.39 0.1 ]
shape of W @ x + b: (5,)
the scores of the 5 neurons: [0.19  0.237 0.17  0.065 0.104]
The largest score is that of neuron 1, whose colour is [0.63 0.46 0.1 ].
```



The middle pixel, and the reference colour with the largest score.

Code: Step 3, the layer of five neurons, and its scores on one pixel cells `layer-of-units`, `many-units-one-point`

```
K = 5
C = reference_colours(X, K)              # K rows and 3 columns: the reference colours, darkest first
W = C.copy()                             # the weights of the layer: row j is reference colour j
b = -0.5 * (C ** 2).sum(axis=1)          # the biases of the layer: one number for each neuron

print("shape of W:", W.shape, "  shape of b:", b.shape)
print("W =")
print(np.round(W, 2))
print("b =", np.round(b, 2))
show_colours(C, [f"neuron {j}" for j in range(K)], "The reference colours: row j of W")
```

OUTPUT

```
shape of W: (5, 3)   shape of b: (5,)
W =
[[0.32 0.36 0.31]
 [0.63 0.46 0.1 ]
 [0.53 0.53 0.45]
 [0.48 0.57 0.66]
 [0.67 0.65 0.55]]
b = [-0.16 -0.31 -0.38 -0.49 -0.59]
```

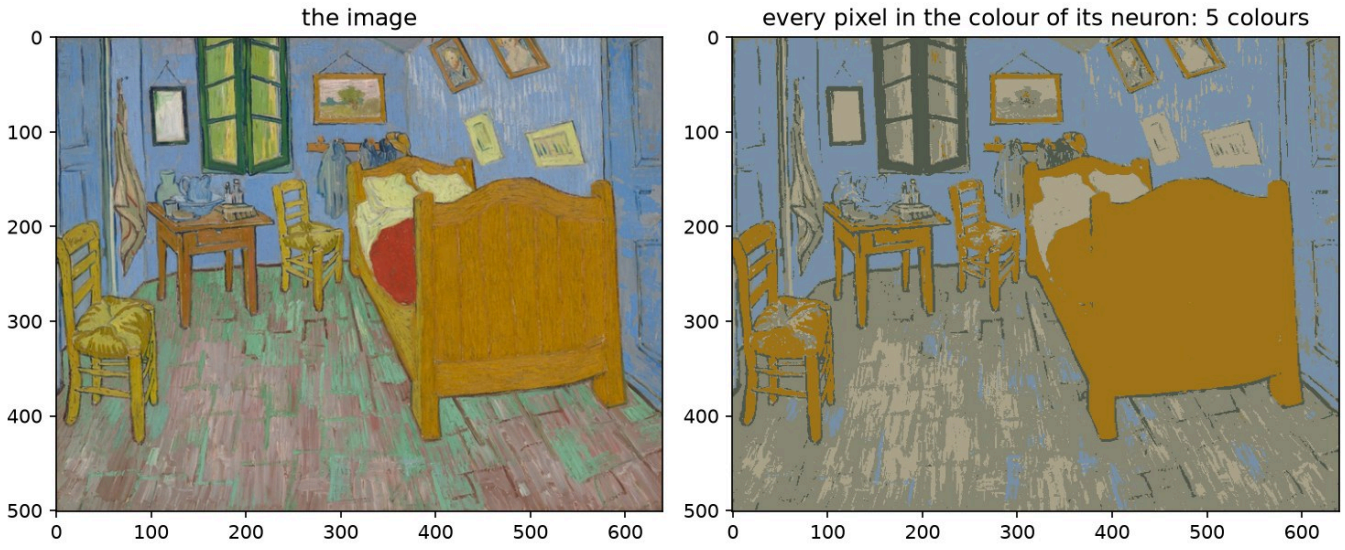
```
s_x = W @ x + b                          # the K scores of the pixel x
print("the pixel x =", np.round(x, 2))
print("shape of W @ x + b:", s_x.shape)
print("the scores of the", K, "neurons:", np.round(s_x, 3))
best = int(s_x.argmax())                 # the position of the largest score
print(f"The largest score is that of neuron {best}, whose colour is {np.round(C[best], 2)}.")
show_colours(np.array([x, C[best]]), ["the pixel x", f"colour of neuron {best}"],
             "The pixel, and the colour with the largest score")
```

1.8 Loop and matrix implementations of the layer Step 4

The scores of every neuron on every point form a matrix `S` with n rows and K columns. `layer_loops` computes them from the definition, with three nested loops: over the points, over the neurons, and over the coordinates. Its innermost line runs $n \times K \times m = 4{,}809{,}600$ times for this image.

Step 4 implements the layer as `X @ W.T + b`. The check compares its result with the three-loop version on the first 2,000 points. Here, omitting the transpose attempts to multiply an $n \times 3$ matrix by a 5×3 matrix, causing a shape error. Locally, the loops took 1.40 seconds and the matrix expression 0.0055 seconds, a time ratio of about 256. The matrix expression performs the arithmetic through optimized compiled operations.

`S.argmax(axis=1)` returns the index of each pixel's largest score. Indexing `C[winner]` then assigns each pixel its nearest reference colour, producing a five-colour image.



Each pixel replaced by its nearest reference colour, selected by the largest score.

Code: Step 4, the three loops, the one line, its check, the measurement, and the argmax

cells `layer-loops`, `paste-layer`, `run-check-layer`, `measure-layer`, `argmax`

```
def layer_loops(X, W, b):
    """The scores of the K neurons of a layer on the n points of X, one multiplication at a time."""
    n, m = X.shape
    K = W.shape[0]
    S = np.empty((n, K))
    for i in range(n):                  # every point
        for j in range(K):              # every neuron
            total = b[j]
            for c in range(m):          # every coordinate
                total += W[j, c] * X[i, c]
            S[i, j] = total
    return S


S_first = layer_loops(X[:2000], W, b)
print("shape of the scores of the first 2,000 points:", S_first.shape)
```

OUTPUT

```
shape of the scores of the first 2,000 points: (2000, 5)
```

```
def layer(X, W, b):
    """The scores of the K neurons of a layer on the n points of X: an array with n rows and K columns."""
    return X @ W.T + b
```

```
try:
    S = layer(X, W, b)
except ValueError:
    raise AssertionError(
        "The shapes do not fit. X has shape (n, 3) and W has shape (K, 3), and the result must have n rows "
        "and K columns. Which array has to be transposed?") from None
assert S is not ..., "Replace the three dots in the function layer with your line, and run that cell again."
S = np.asarray(S)
assert S.shape == (n, K), (
    f"layer returned an array of shape {S.shape}. It must be {(n, K)}: one row for each point and one column for each
neuron.")
assert np.allclose(S[:2000], S_first), (
    "The scores differ from those of layer_loops. Check which array is transposed, and that the bias is added.")
print(f"Correct: layer(X, W, b) has shape {S.shape} and agrees with the three loops.")
```

OUTPUT

```
Correct: layer(X, W, b) has shape (320640, 5) and agrees with the three loops.
```

```
start = time.perf_counter()
S_loops = layer_loops(X, W, b)
t_loops = time.perf_counter() - start

start = time.perf_counter()
for _ in range(20):
    S = layer(X, W, b)
t_product = (time.perf_counter() - start) / 20
```

```
print(f"n = {n:,} points, m = {m} coordinates, K = {K} neurons")
print(f"The innermost line of the loops runs n x K x m = {n * K * m:,} times.")
print(f"{'version':16s}{'seconds':>12s}")
print(f"{'three loops':16s}{t_loops:12.4f}")
print(f"{'one product':16s}{t_product:12.4f}")
print(f"The loops take {t_loops / t_product:.0f} times as long as the product.")
```

OUTPUT

```
n = 320,640 points, m = 3 coordinates, K = 5 neurons
The innermost line of the loops runs n x K x m = 4,809,600 times.
version              seconds
three loops           1.4020
one product           0.0055
The loops take 256 times as long as the product.
```

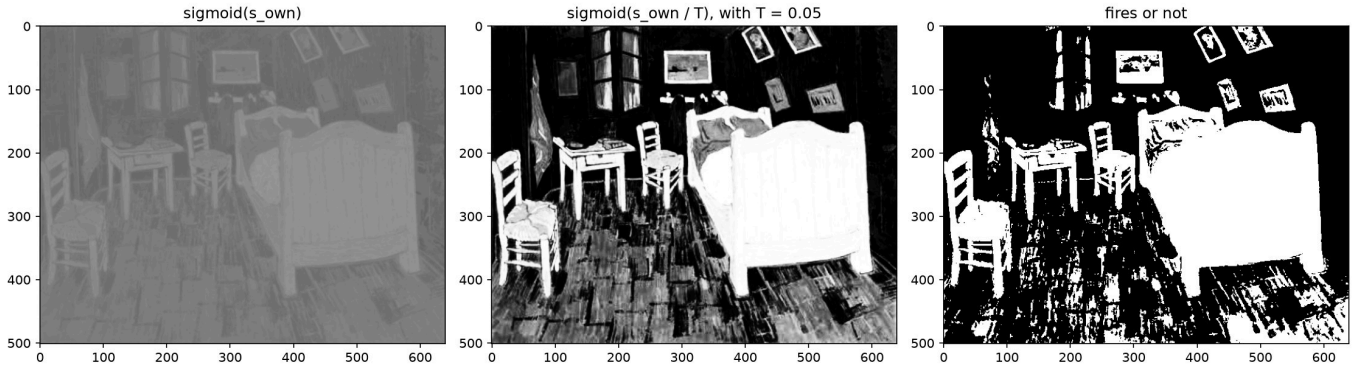
```
winner = S.argmax(axis=1)                # for each point, the neuron with the largest score
print("shape of winner:", winner.shape, "  its first ten entries:", winner[:10])
shares = np.bincount(winner, minlength=K) / n
for j in range(K):
    print(f"neuron {j}, colour {np.round(C[j], 2)}: the largest score on {shares[j]:.1%} of the pixels")
show([image, C[winner].reshape(height, width, 3)],
     ["the image", f"every pixel in the colour of its neuron: {K} colours"])
```

OUTPUT

```
shape of winner: (320640,)   its first ten entries: [2 2 2 2 2 2 2 2 2 2]
neuron 0, colour [0.32 0.36 0.31]: the largest score on 7.8% of the pixels
neuron 1, colour [0.63 0.46 0.1 ]: the largest score on 23.0% of the pixels
neuron 2, colour [0.53 0.53 0.45]: the largest score on 33.1% of the pixels
neuron 3, colour [0.48 0.57 0.66]: the largest score on 24.4% of the pixels
neuron 4, colour [0.67 0.65 0.55]: the largest score on 11.7% of the pixels
```

1.9 The sigmoid on every pixel Step 5

`1 / (1 + np.exp(-s))` applies sigmoid entry by entry. For the Step 2 neuron, the outputs range from 0.39 to 0.60 because the scores are near zero. Dividing by $T = 0.05$ multiplies the scores by 20, producing values closer to 0 and 1. As $T \to 0$, $\sigma(s / T)$ approaches 1 for $s > 0$ and 0 for $s < 0$; at $s = 0$ it remains 0.5. This approximates the threshold decision except at zero.



Sigmoid at temperatures 1 and 0.05, followed by the threshold mask s >= 0.

Code: Step 5, the sigmoid cell `sigmoid`

```
def sigmoid(s):
    """The sigmoid of every entry of the array s."""
    return 1 / (1 + np.exp(-s))


s_own = X @ w_own + b_own                # the scores of your own neuron of Step 2 on all the points
T = 0.05                                 # the temperature

print("shape of s_own:", s_own.shape, " shape of sigmoid(s_own):", sigmoid(s_own).shape)
print("shape of S:    ", S.shape, " shape of sigmoid(S):    ", sigmoid(S).shape)
print(f"sigmoid(s_own) lies between {sigmoid(s_own).min():.2f} and {sigmoid(s_own).max():.2f}")
print(f"sigmoid(s_own / T) lies between {sigmoid(s_own / T).min():.2f} and {sigmoid(s_own / T).max():.2f}")
show([sigmoid(s_own).reshape(height, width), sigmoid(s_own / T).reshape(height, width),
      (s_own >= 0).reshape(height, width).astype(float)],
     ["sigmoid(s_own)", f"sigmoid(s_own / T), with T = {T}", "fires or not"])
```

OUTPUT

```
shape of s_own: (320640,)  shape of sigmoid(s_own): (320640,)
shape of S:     (320640, 5)  shape of sigmoid(S):     (320640, 5)
sigmoid(s_own) lies between 0.39 and 0.60
sigmoid(s_own / T) lies between 0.00 and 1.00
```

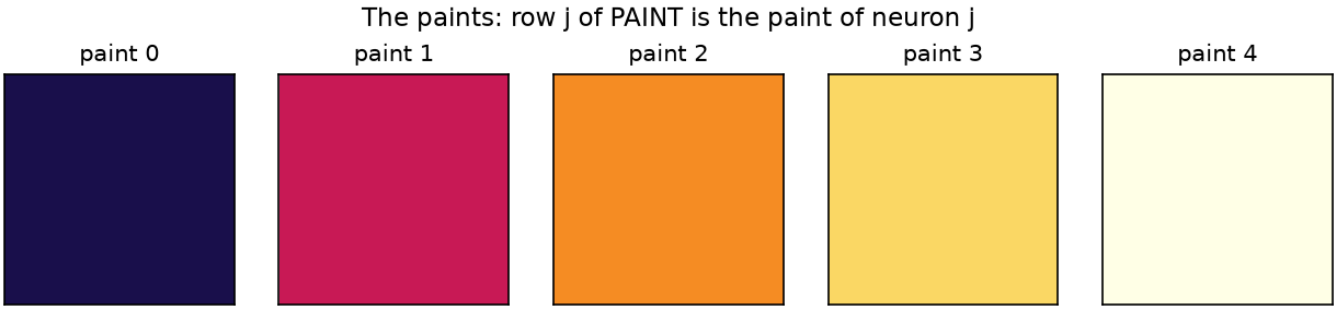
1.10 Softmax weights and image recolouring Steps 6 and 7

`softmax_rows(S / T)` converts each pixel's five scores into probability weights. For the displayed pixel, the scores differ by a few hundredths. At $T = 0.02$, these differences are multiplied by 50, concentrating weight on the largest score:

OUTPUT

```
the scores of the point in row 0:        [0.295 0.29  0.324 0.279 0.29 ]
its probabilities at temperature 0.02:  [0.137 0.109 0.586 0.062 0.107]
the sum of its probabilities: 1.0
```

Step 7 computes the recoloured image with `Prob @ PAINT`, as in point 7. Each output pixel is a weighted average of the five paints. At temperature 1, the weights are close to 1 / 5 in this example, producing a nearly uniform colour. At 0.002, most pixels are dominated by the paint assigned to their nearest reference colour, producing sharper regions. Pixels near score ties can still mix paints.



The notebook's five paints, the rows of PAINT.

Temperature changes the mixture weights while keeping the palette fixed, so it can change both contrast and the resulting pixel colours. Lower temperatures make the largest score dominate, producing sharper colour regions.

Code: Steps 6 and 7, the probabilities and the repainting

cells `probabilities` , `repaint`

```
T = 0.02                                # the temperature of the softmax
Prob = softmax_rows(S / T)              # n rows and K columns: the probabilities
print("shape of Prob:", Prob.shape)
print("the scores of the point in row 0:       ", np.round(S[0], 3))
print(f"its probabilities at temperature {T}: ", np.round(Prob[0], 3))
print("the sum of its probabilities:", round(float(Prob[0].sum()), 6))
```

OUTPUT

```
shape of Prob: (320640, 5)
the scores of the point in row 0:         [0.295 0.29  0.324 0.279 0.29 ]
its probabilities at temperature 0.02:  [0.137 0.109 0.586 0.062 0.107]
the sum of its probabilities: 1.0
```

```
PAINT = np.array([[0.10, 0.07, 0.30],      # the paint of neuron 0, the neuron of the darkest reference colour
                  [0.80, 0.10, 0.35],      # the paint of neuron 1
                  [0.98, 0.55, 0.15],      # the paint of neuron 2
                  [0.99, 0.85, 0.40],      # the paint of neuron 3
                  [1.00, 1.00, 0.92]])     # the paint of neuron 4, the neuron of the lightest reference colour
T = 0.02                                   # the temperature of the softmax

assert PAINT.shape == (K, 3), f"PAINT must have {K} rows, one for each neuron, and 3 columns."
Prob = softmax_rows(layer(X, W, b) / T)    # n rows and K columns
new = Prob @ PAINT                         # n rows and 3 columns
print("shape of Prob:", Prob.shape, "  shape of PAINT:", PAINT.shape, "  shape of new:", new.shape)

show_colours(PAINT, [f"paint {j}" for j in range(K)], "The paints: row j of PAINT is the paint of neuron j")
show([image, new.reshape(height, width, 3)], ["the image", f"repainted, at temperature {T}"])

paints = "; ".join(str(tuple(float(v) for v in np.round(paint, 2))) for paint in PAINT)
print("Line for the discussion board:")
print(f"Image {IMAGE} ({picture_info['title']}), repainted with {K} paints at temperature {T}. "
      f"Paints (red, green, blue): {paints}.")
```

OUTPUT

```
shape of Prob: (320640, 5)   shape of PAINT: (5, 3)   shape of new: (320640, 3)
Line for the discussion board:
Image 7 (The Bedroom), repainted with 5 paints at temperature 0.02. Paints (red, green, blue): (0.1, 0.07,
0.3); (0.8, 0.1, 0.35); (0.98, 0.55, 0.15); (0.99, 0.85, 0.4); (1.0, 1.0, 0.92).
```

Act 2. PyTorch and GPU computation Open the notebook of Act 2

PyTorch provides tensor operations, neural-network layers, and automatic differentiation. This notebook implements the colour layer in PyTorch, moves its tensors to a GPU, and compares CPU and GPU timings. Select a GPU runtime before starting; changing the runtime can reset the session. The notebook also supports

CPU-only sessions.

2.1 From arrays to tensors Step 10

PyTorch uses **tensor** for an array with any number of dimensions. The examples below show shapes used in the notebooks.

Name	Dimensions	Example in this meeting	Shape
scalar	0	the bias of one neuron	a single number
vector	1	one pixel; the weights of one neuron	`(3,)`
matrix	2	the points `X`; the weights `W` of a layer	`(n, 3)`, `(K, 3)`
three-dimensional array	3	a colour image	`(height, width, 3)`
four-dimensional array	4	a batch of colour images	`(30, 96, 96, 3)`

`torch.from_numpy` preserves a supported NumPy array's data type and shares its storage: changing one also changes the other. `torch.tensor` copies its input and infers a data type unless one is specified. For a list of Python floats, it uses the configured floating-point default.

Code: Step 10, the layer of Act 1 again, and three tensors cells `image-number`, `tensors`

```
IMAGE = 7        # replace 0 with your group's number, from 1 to 30

assert 1 <= IMAGE <= 30, "Type your group's number in place of the 0."

images = pd.read_csv(IMAGES + "images.csv", index_col="number")
picture_info = images.loc[IMAGE]
image = read_image(picture_info["file"])
height, width = image.shape[0], image.shape[1]

X = image.reshape(-1, 3)                # the points: one row for each pixel
n, m = X.shape
K = 5
C = reference_colours(X, K)             # the reference colours
W = C.copy()                            # the weights of the layer: one row for each neuron
b = -0.5 * (C ** 2).sum(axis=1)         # the biases of the layer: one number for each neuron
S = X @ W.T + b                         # the scores: the line of Act 1

print(f"Image {IMAGE}: {picture_info['title']} ({picture_info['creator']})")
print("NumPy arrays.  X:", X.shape, "  W:", W.shape, "  b:", b.shape, "  S:", S.shape)
```

OUTPUT

```
Image 7: The Bedroom (Vincent van Gogh)
NumPy arrays.  X: (320640, 3)   W: (5, 3)   b: (5,)   S: (320640, 5)
```

```
w_t = torch.tensor([0.299, 0.587, 0.114])        # the weights of the grey neuron, from a list
X_t = torch.from_numpy(X)                        # the points, from the NumPy array X
image_t = torch.from_numpy(image)                # the image, from the NumPy array image

for name, tensor in [("w_t", w_t), ("X_t", X_t), ("image_t", image_t)]:
    print(f"{name:8s} dimensions: {tensor.ndim}   shape: {tuple(tensor.shape)}   type of its numbers: {tensor.dtype}")
```

OUTPUT

```
w_t      dimensions: 1   shape: (3,)   type of its numbers: torch.float32
X_t      dimensions: 2   shape: (320640, 3)   type of its numbers: torch.float64
image_t  dimensions: 3   shape: (501, 640, 3)   type of its numbers: torch.float64
```

2.2 Floating-point data types Step 10

The NumPy arrays here use `float64`: 8 bytes per value and about 16 significant decimal digits. PyTorch's initial floating-point default is `float32`: 4 bytes and about 7 digits. Both libraries support other types. Smaller types reduce storage and may increase throughput on suitable hardware, at the cost of precision or range. In this example, `X_t @ w_t` fails because its operands have different types. `X_t.float()` converts the points to match the float32 weights, reducing their storage to 3.8 MB from 7.7 MB.

Code: Step 10, two types, and the conversion cell `dtype`

```python
try:
    X_t @ w_t
except RuntimeError as error:
    print("X_t @ w_t fails:", error)

X_t = X_t.float()                                # the same numbers, as float32
megabytes = X_t.element_size() * X_t.nelement() / 1e6
print("after X_t = X_t.float(), the type of its numbers is", X_t.dtype)
print(f"memory of the points: {X.nbytes / 1e6:.1f} MB as float64, {megabytes:.1f} MB as float32")

s_t = X_t @ w_t                                  # one neuron, many points: the line of Act 1
print("shape of X_t @ w_t:", tuple(s_t.shape))
```

OUTPUT

```
X_t @ w_t fails: addmv input tensors must have the same dtype, but got Float, Double, and Float
after X_t = X_t.float(), the type of its numbers is torch.float32
memory of the points: 7.7 MB as float64, 3.8 MB as float32
shape of X_t @ w_t: (320640,)
```

2.3 The layer in PyTorch, and the softmax translated Step 11

`X_t @ W_t.T + b_t` computes the layer's scores in PyTorch. The largest difference from the float64 NumPy result is `1.8e-07`. Float32 introduces rounding in both the inputs and the arithmetic. For normal-range values rounded to nearest, a single conversion has relative error at most about $2^{-24} \approx 6 \times 10^{-8}$; this is not a bound on the error of an entire matrix product.

	NumPy	PyTorch
an array from a list	`np.array([1.0, 2.0])`	`torch.tensor([1.0, 2.0])`
the layer	`X @ W.T + b`	`X @ W.T + b`
a function of every entry	`np.exp(S)`	`torch.exp(S)`
reduce across each row	`axis=1` , `keepdims=True`	`dim=1` , `keepdim=True`
the largest entry of each row	`S.max(axis=1)` , an array	`S.max(dim=1)` , with `.values` and `.indices`
initial default for Python floats	`float64`	`float32`
where the numbers are	the main memory	a **device**: the CPU or a GPU

Step 11 asks for a PyTorch version of `softmax_rows` with the same input and output shape, using neither loops, NumPy, nor the built-in softmax. Replace `np.exp` with `torch.exp`, use `dim` and `keepdim`, and extract `.values` from `S.max(dim=1, keepdim=True)`. Keeping the reduced dimension gives row maxima of shape `(n, 1)`. Without it, shape `(n,)` aligns with the last dimension during broadcasting, usually causing a shape error or an incorrect subtraction. Using PyTorch operations also lets the function run on GPU tensors. The check compares its result with `torch.softmax(S, dim=1)`.

Code: Step 11, the layer in PyTorch, the translated softmax, and its check

cells `layer-torch` , `paste-softmax-torch` , `run-check-softmax-torch` , `repaint-torch`

```
W_t = torch.from_numpy(W).float()        # the weights of the layer, as a float32 tensor
b_t = torch.from_numpy(b).float()        # the biases of the layer

S_t = X_t @ W_t.T + b_t                  # the line of Act 1, unchanged
print("shape of S_t:", tuple(S_t.shape))
print(f"largest difference from the NumPy scores S: {np.abs(S_t.numpy() - S).max():.1e}")

winner_t = S_t.argmax(dim=1)                    # NumPy: S.argmax(axis=1)
largest_t = S_t.max(dim=1, keepdim=True)        # NumPy: S.max(axis=1, keepdims=True)
print("shape of S_t.argmax(dim=1):", tuple(winner_t.shape))
print("S_t.max(dim=1, keepdim=True) returns two tensors: .values of shape", tuple(largest_t.values.shape),
      "and .indices of shape", tuple(largest_t.indices.shape))
print("torch.sigmoid of the first row of scores:", torch.sigmoid(S_t[0]))
```

OUTPUT

```
shape of S_t: (320640, 5)
largest difference from the NumPy scores S: 1.8e-07
shape of S_t.argmax(dim=1): (320640,)
S_t.max(dim=1, keepdim=True) returns two tensors: .values of shape (320640, 1) and .indices of shape (320640,
1)
torch.sigmoid of the first row of scores: tensor([0.5732, 0.5721, 0.5803, 0.5693, 0.5720])
```

```
def softmax_rows_torch(S):
    shifted = S - S.max(dim=1, keepdim=True).values  # the largest entry of every row becomes 0
    E = torch.exp(shifted)
    return E / E.sum(dim=1, keepdim=True)
```

```
assert "softmax_rows_torch" in globals(), "Paste the function softmax_rows_torch into the cell above and run that cell."
assert not has_loop(softmax_rows_torch), "This function must use operations on whole tensors: no loop."
assert not mentions(softmax_rows_torch, ("softmax(", "Softmax", "scipy", "sklearn")), (
```

```
    "This function must compute the softmax itself: not torch.softmax, and no softmax function from another library.")
assert not mentions(softmax_rows_torch, ("np.", "numpy")), (
    "This function must compute with PyTorch only: no NumPy function, and no conversion to a NumPy array. "
    "NumPy cannot compute with a tensor that is on a GPU.")

test = torch.tensor([[-1.0, 0.0, 2.0, 1.0],
                     [0.0, 0.0, 0.0, 0.0],
                     [1000.0, 1003.0, 1001.0, 1002.0]])
try:
    result = softmax_rows_torch(test.clone())
except TypeError:
    raise AssertionError(
        "A tensor was combined with something that is not a tensor. In PyTorch, S.max(dim=1, keepdim=True) returns "
        "two things, the largest values and their positions: take its .values.") from None
except RuntimeError:
    raise AssertionError(
        "The shapes inside softmax_rows_torch do not fit. The largest entry and the sum of every row must keep "
        "the shape (n, 1): use keepdim=True.") from None
assert isinstance(result, torch.Tensor), "softmax_rows_torch must return a PyTorch tensor."
assert result.shape == test.shape, f"softmax_rows_torch returned a tensor of shape {tuple(result.shape)}; it must be
{tuple(test.shape)}."
assert not torch.isnan(result).any(), (
    "softmax_rows_torch returns nan (not a number) for the row of very large scores. "
    "Subtract the largest entry of each row before the exponential.")
assert torch.allclose(result.sum(dim=1), torch.ones(3)), (
    "The rows of the result do not add to 1. The largest entry and the sum must be taken along each row: dim=1.")
assert torch.allclose(result, torch.softmax(test, dim=1), atol=1e-5), (
    f"The result is not the softmax of the rows. For the scores -1, 0, 2, 1 the function gives {result[0].tolist()}; "
    "the right values are 0.03, 0.09, 0.64, 0.24.")
print("Correct: softmax_rows_torch agrees with PyTorch's own torch.softmax(S, dim=1), also for very large scores.")
```

OUTPUT

```
Correct: softmax_rows_torch agrees with PyTorch's own torch.softmax(S, dim=1), also for very large scores.
```

```
# the paints of the five neurons: replace these rows with the paints of your group
PAINT = np.array([[0.10, 0.07, 0.30],
                  [0.80, 0.10, 0.35],
                  [0.98, 0.55, 0.15],
                  [0.99, 0.85, 0.40],
                  [1.00, 1.00, 0.92]])
T = 0.02                                  # the temperature of the softmax

PAINT_t = torch.from_numpy(PAINT).float()
Prob_t = softmax_rows_torch((X_t @ W_t.T + b_t) / T)     # scores, then probabilities
new_t = Prob_t @ PAINT_t                                  # probabilities, then colours
print("shape of new_t:", tuple(new_t.shape), "  type of its numbers:", new_t.dtype)
show([image, new_t.reshape(height, width, 3).numpy()], ["the image", "repainted with PyTorch"])
```

OUTPUT

```
shape of new_t: (320640, 3)   type of its numbers: torch.float32
```

2.4 The built-in linear layer Step 12

`torch.nn.Linear(3, K)` creates an affine layer with randomly initialized weights and biases. The weights have shape `(K, 3)`, and the transformation is $y = xA^T + b$. Copying in this example's parameters reproduces `X_t @ W_t.T + b_t`, up to floating-point differences. During gradient-based training, a **loss** measures prediction error, and its **gradient** gives derivatives with respect to the parameters. PyTorch can compute these derivatives automatically. The example uses `torch.no_grad()` because it only assigns parameters and evaluates the layer.

Code: Step 12, torch.nn.Linear cell `linear`

```
linear = torch.nn.Linear(3, K)           # a layer of K neurons for points with 3 coordinates
print("shape of linear.weight:", tuple(linear.weight.shape))
print("shape of linear.bias:  ", tuple(linear.bias.shape))

with torch.no_grad():                    # this cell needs no gradients
    linear.weight.copy_(W_t)             # the weights of Act 1 go into the layer
    linear.bias.copy_(b_t)               # and so do the biases
    S_linear = linear(X_t)               # the layer, applied to all the points

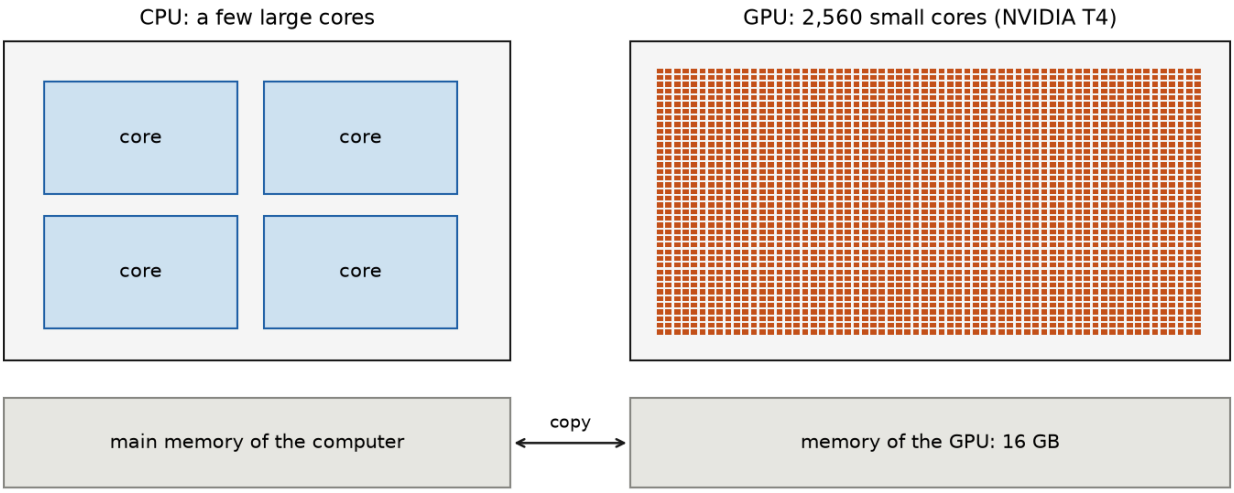
print("shape of linear(X_t):", tuple(S_linear.shape))
difference = float((S_linear - S_t).abs().max())
print(f"largest difference between linear(X_t) and X_t @ W_t.T + b_t: {difference:.1e}")
```

OUTPUT

```
shape of linear.weight: (5, 3)
shape of linear.bias:   (5,)
shape of linear(X_t): (320640, 5)
largest difference between linear(X_t) and X_t @ W_t.T + b_t: 0.0e+00
```

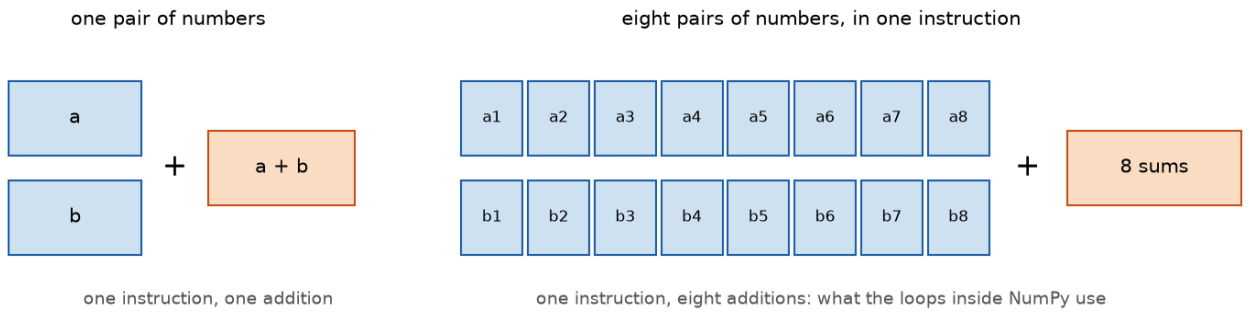
2.5 A second processor: the GPU Step 13

A **CPU**, or central processing unit, executes general-purpose programs using one or more **cores**. CPU cores support control flow, caching, and parallel instruction execution. A **GPU**, or graphics processing unit, provides many arithmetic units for workloads with substantial parallelism. Threads execute in groups that can apply the same instruction to different data. The NVIDIA T4 used in the class sessions has 2,560 CUDA cores, 16 GB of memory, a peak float32 rate of 8.1 trillion operations per second, and a 70-watt power rating. CUDA cores are arithmetic units, so their count is not directly comparable with the number of CPU cores.



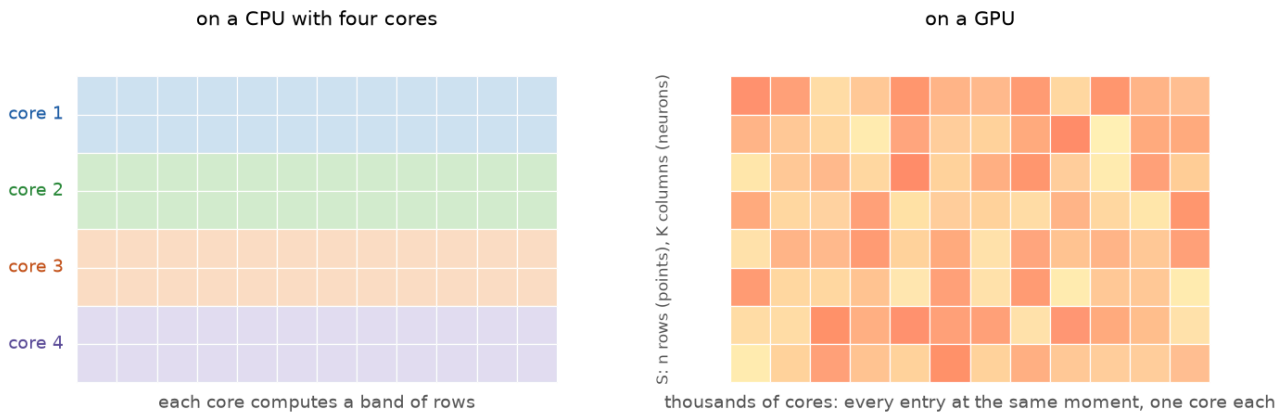
Two processors, each computing with the numbers of its own memory.

CPUs also exploit parallelism. **Vector instructions** apply an arithmetic operation to several values at once; the diagram shows eight additions. NumPy and PyTorch implementations can use these instructions. A **cache** holds recently used data near the core, reducing the cost of repeated memory access. Multiple CPU cores can also share the work of a large matrix product.



Scalar and vector addition; the vector width shown is eight values.

Each output entry of a matrix product is a sum over the input coordinates and can be computed independently of the other output entries. Implementations divide this work among processing units, often in blocks that reuse input data. The diagram shows a row partition for a CPU and many concurrent entries for a GPU. Available parallelism, memory access, and scheduling determine how much speedup an implementation achieves.



One product divided among the cores of a CPU and of a GPU.

Code: Step 13, the processors of this session cell `gpu-facts`

```
print("logical cores of the CPU that this session can use:", os.cpu_count())
if GPU:
    gpu = torch.cuda.get_device_properties(0)
    print("the GPU:", gpu.name)
    print(f"memory of the GPU: {gpu.total_memory / 2 ** 30:.1f} GB")
else:
    print("This session has no GPU. Read these numbers in the notebook of a member whose session has one.")
```

OUTPUT

```
logical cores of the CPU that this session can use: 32
This session has no GPU. Read these numbers in the notebook of a member whose session has one.
```

2.6 Moving tensors between devices Step 14

PyTorch uses `"cuda"` for NVIDIA GPU devices. Calling `X_t.to("cuda")` on a CPU tensor returns a GPU copy. The matrix operands must share a device, so `X_g @ W_t.T` raises an error when only `X_g` is on the GPU. Once the points, weights, and biases are on the GPU, the same layer expression runs there. Calling `.cpu()` on the result returns a CPU copy. If a tensor is already on the requested device with the requested type, these conversions need not make a copy.

The output below comes from the local CPU-only session. In a GPU session, the cell prints each tensor's device, demonstrates the mixed-device error, and compares CPU and GPU scores.

Transfers between CPU and GPU memory take time. They can outweigh the benefit of accelerating a short computation. Keeping data on the GPU for repeated operations reduces those transfers, provided the data fit in its memory. Work with little parallelism may also gain little from a GPU. CPU and GPU, Part 4 discusses these tradeoffs. In a T4 session, the notebook times a copy of a 4,096 × 4,096 float32 matrix, about 67 MB, alongside the product. No class post included that copy measurement.

Code: Step 14, tensors on two devices cell `device`

```
print("X_t is on the device:", X_t.device)
if GPU:
    X_g = X_t.to("cuda")                # copies of the three tensors, in the memory of the GPU
    W_g = W_t.to("cuda")
    b_g = b_t.to("cuda")
    print("X_g is on the device:", X_g.device)

    try:
        X_g @ W_t.T                     # X_g is on the GPU and W_t is on the CPU
    except RuntimeError as error:
        print("X_g @ W_t.T fails:", error)

    S_g = X_g @ W_g.T + b_g             # the line of Act 1, carried out by the GPU
    print("S_g is on the device:", S_g.device)
    difference = float((S_g.cpu() - S_t).abs().max())
    print(f"largest difference between the scores of the GPU and of the CPU: {difference:.1e}")
else:
    print("This session has no GPU, so every tensor stays in the memory of the CPU.")
```

OUTPUT

```
X_t is on the device: cpu
This session has no GPU, so every tensor stays in the memory of the CPU.
```

2.7 Measuring the product on both processors Step 15

The image's colour layer has only five neurons and three input coordinates. To test a larger workload, `time_product(size, device)` uses $n = m = K = \text{size}$ with random float32 inputs already on the target device. The timed interval excludes data transfers. At size 4,096, the standard product performs 4096^3, about 69 billion, multiplications. A warm-up product runs before measurement. For GPU timings, `torch.cuda.synchronize()` waits for queued work before the timer starts and for the measured work before it stops; otherwise a CPU clock can underreport execution time.

Code: Step 15, the timer cell `time-product`

```
def finish(device):
    """Wait until the device has finished its work. A GPU may still be working when PyTorch goes on."""
    if device == "cuda":
        torch.cuda.synchronize()

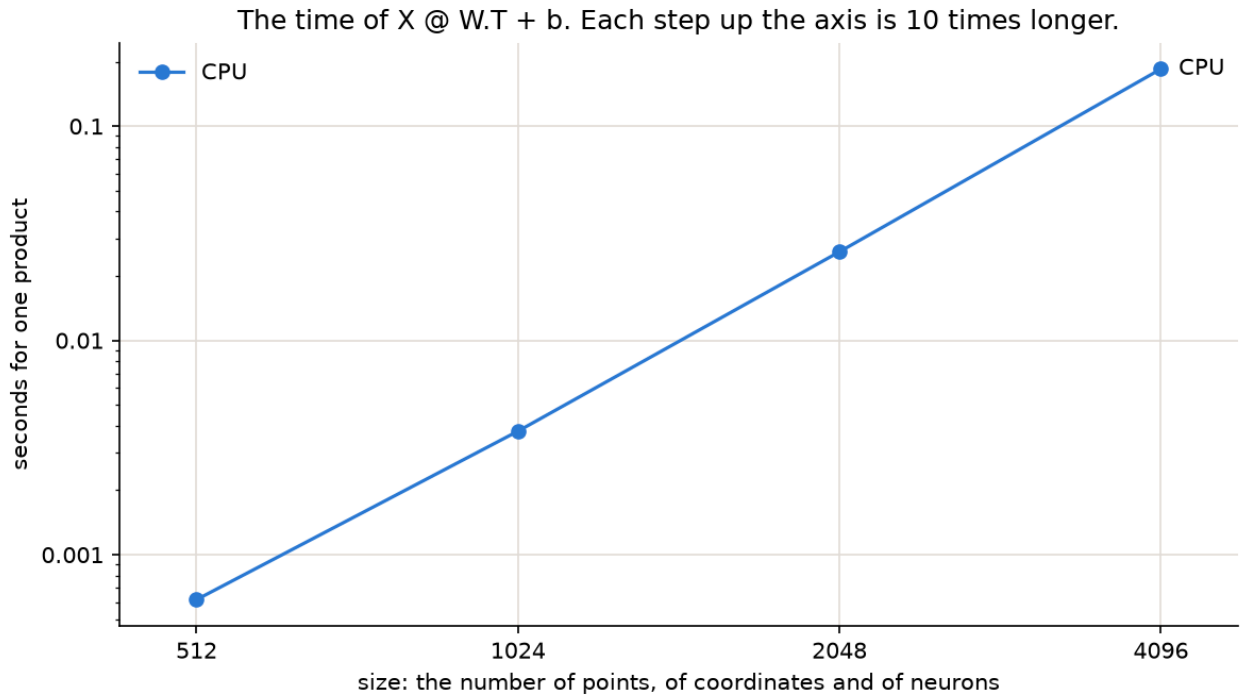

def time_product(size, device):
    """The average time, in seconds, of X @ W.T + b on the device, with n = m = K = size."""
    X = torch.rand(size, size, device=device)        # random float32 numbers, made on the device
    W = torch.rand(size, size, device=device)
    b = torch.rand(size, device=device)
    repeats = max(1, min(200, 8_000_000_000 // size ** 3))     # many repeats when the product is small
    finish(device)
    start = time.perf_counter()
    for _ in range(repeats):
        S = X @ W.T + b
```

```
    finish(device)
    return (time.perf_counter() - start) / repeats


def warm_up(device):
    """One small product that is not measured."""
    time_product(64, device)


print("time_product and warm_up are defined.")
```

On the local machine, with 32 logical processors, the product at size 4,096 took 0.18 seconds. Both chart axes are logarithmic: successive tested sizes double, and labelled vertical ticks differ by factors of ten. A power law $t \approx c\,\text{size}^p$ appears as a straight line on log-log axes. Its slope in log coordinates is p; the apparent angle also depends on the plot's scaling.



Local CPU timings. Running the notebook with a GPU adds its timing curve.

Code: Step 15, the measurement and the chart

cells `measure-devices`, `timing-chart`

```
sizes = [512, 1024, 2048, 4096]
devices = ["cpu", "cuda"] if GPU else ["cpu"]
seconds = {}
for device in devices:
    warm_up(device)
    seconds[device] = [time_product(size, device) for size in sizes]

gpu_name = torch.cuda.get_device_name(0) if GPU else "none"
print(f"X @ W.T + b with n = m = K = size, in seconds. CPU: {os.cpu_count()} logical cores. GPU: {gpu_name}.")
print(f"{'size':>6s}{'multiplications':>18s}{'CPU':>12s}" + (f"{'GPU':>12s}{'CPU / GPU':>12s}" if GPU else ""))
for i, size in enumerate(sizes):
```

```
    line = f"{size:6d}{size ** 3:18,d}{seconds['cpu'][i]:12.6f}"
    if GPU:
        line += f"{seconds['cuda'][i]:12.6f}{seconds['cpu'][i] / seconds['cuda'][i]:12.1f}"
    print(line)

if GPU:
    X_cpu = torch.rand(4096, 4096)
    finish("cuda")
    start = time.perf_counter()
    X_gpu = X_cpu.to("cuda")
    finish("cuda")
    t_move = time.perf_counter() - start
    megabytes = X_cpu.element_size() * X_cpu.nelement() / 1e6
    print(f"\nCopying one matrix of size 4096 ({megabytes:.0f} MB) to the GPU takes {t_move:.4f} seconds; "
          f"the product at that size takes the GPU {seconds['cuda'][-1]:.4f} seconds.")
```

OUTPUT

```
X @ W.T + b with n = m = K = size, in seconds. CPU: 32 logical cores. GPU: none.
  size   multiplications         CPU
   512       134,217,728    0.000617
  1024     1,073,741,824    0.003772
  2048     8,589,934,592    0.025927
  4096    68,719,476,736    0.184849
```

```
fig, ax = plt.subplots(figsize=(8, 4.6))
ax.plot(sizes, seconds["cpu"], marker="o", color="#2a78d6", label="CPU")
ax.annotate("CPU", (sizes[-1], seconds["cpu"][-1]), xytext=(8, 0), textcoords="offset points", va="center")
if GPU:
    ax.plot(sizes, seconds["cuda"], marker="s", linestyle="--", color="#eb6834", label="GPU")
    ax.annotate("GPU", (sizes[-1], seconds["cuda"][-1]), xytext=(8, 0), textcoords="offset points", va="center")
ax.set_xscale("log", base=2)
ax.set_yscale("log")
ax.set_xticks(sizes)
ax.set_xticklabels([str(size) for size in sizes])
ax.yaxis.set_major_formatter(plt.FuncFormatter(lambda value, position: f"{value:g}"))
ax.margins(x=0.08)
ax.set_xlabel("size: the number of points, of coordinates and of neurons")
ax.set_ylabel("seconds for one product")
ax.set_title("The time of X @ W.T + b. Each step up the axis is 10 times longer.")
ax.grid(True, which="major", color="#e1e0d9", linewidth=0.8)
ax.spines[["top", "right"]].set_visible(False)
ax.legend(frameon=False)
fig.tight_layout()
plt.show()

last, first = len(sizes) - 1, 0
print("Line for the discussion board:")
if GPU:
    print(f"GPU {gpu_name}, CPU with {os.cpu_count()} logical cores. Size 4096: CPU {seconds['cpu'][last]:.3f} s, "
          f"GPU {seconds['cuda'][last]:.4f} s, so the GPU is {seconds['cpu'][last] / seconds['cuda'][last]:.0f} times as
fast. "
          f"Size {sizes[first]}: CPU {seconds['cpu'][first] * 1e3:.1f} milliseconds, GPU {seconds['cuda'][first] *
1e3:.2f} milliseconds.")
else:
    print(f"No GPU in this session. CPU with {os.cpu_count()} logical cores. Size 4096: {seconds['cpu'][last]:.3f} s. "
          f"Size {sizes[first]}: {seconds['cpu'][first] * 1e3:.1f} milliseconds.")
```

OUTPUT

```
Line for the discussion board:
No GPU in this session. CPU with 32 logical cores. Size 4096: 0.185 s. Size 512: 0.6 milliseconds.
```

GPU speedup need not increase at every matrix size. One complete measurement table reported speedups of 24, 34, 33, and 35 as size increased from 512 to 4,096. These timings exclude data transfers, so transfer costs cannot explain differences within this experiment.

Doubling all three matrix dimensions multiplies the standard product's multiplication count by eight. Timing ratios can differ because throughput changes with size. This growth is cubic; exponential growth such as 2^n instead multiplies by a fixed factor whenever n increases by one.